

Turing Makinesi ile İkili (Binary) Sayı Çarpma Proje Raporu

Semih Eren ÖZEN

23060535

<https://github.com/SemihOzen/Ozdevinim-Kurami-ODEV1>

https://youtu.be/i5wzkig_AMA

1. Problem Tanımı

Bu projenin amacı, tek bantlı (single-tape) bir Turing Makinesi simülatörü geliştirerek, verilen iki ikili (binary) sayının çarpımını hesaplamaktır. Makine, işlemleri *Kaydır ve Topla* (Shift & Add) algoritmasına dayanarak, saf bir Turing Makinesi mantığıyla bant üzerinde fiziksel sembol manipülasyonu yaparak gerçekleştirmelidir.

2. Turing Makinesi Modeli

Projede genişleyebilir tek bantlı (single-tape) Turing makinesi kullanılmıştır. Sistem şu bileşenlerden oluşmaktadır:

- **Q (Durumlar Kümesi):** $q_{\text{baslangic}}$, $q_{\text{yildiz_bulundu}}$, $q_{\text{isle_bit_k}}$, $q_{\text{shift_...}}$, $q_{\text{yaz_...}}$, q_{kabul} , q_{red} vb.
- **Σ (Giriş Alfabeti):** $\{'0', '1', '*', '='\}$
- **Γ (Bant Alfabeti):** $\{'0', '1', '*', '=', 'B', 'X', 'Y'\}$ (Buradaki 'B' boşluk sembolüdür).
- **δ (Geçiş Fonksiyonu):** Durumlar arası geçişleri (Yazılacak Sembol, Sonraki Durum, Kafa Hareketi) belirten sözlük yapısı.
- **q_0 (Başlangıç Durumu):** $q_{\text{baslangic}}$
- **F (Kabul Durumu):** q_{kabul}

3. Binary Sayı Sistemi

Makine sadece '0' ve '1' rakamlarından oluşan ikili sistem sayılarını kabul etmektedir. Başlangıçta bant üzerine yerleştirilen bu sayılar, algoritmada sırasıyla sağdan sola (en değersiz bitten en değerli bite doğru) işlenir. Çarpma algoritmasının gereği olarak, 2 tabanındaki her basamak kaydırması, birinci sayının bant üzerinde 2 ile çarpılmasını (sonuna '0' eklenmesini) fiziksel olarak simüle eder.

4. Operand Ayırıştırma Yaklaşımı

Tasarımın en kritik kısımlarından biri operandların doğru şekilde ayrıştırılmasıdır:

- Bant üzerine girdi $SAYI1 * SAYI2 =$ formatında yerleştirilir.
- Makine ilk iş olarak $*$ işaretini bulup `q_yildiz_bulundu` durumuna geçerek sayının solunu **çarpılan (multiplicand)**, sağını ise **çarpan (multiplier)** olarak beynine kazır.
- İşlem sırasında her iki sayı da okunurken veya kaydırılırken kafanın yön bulması için bu $*$ sembolü aktif bir sınır (landmark) olarak kullanılır.
- Çarpan bitleri işlendikten sonra makine daima $*$ sembolüne geri dönerek konumunu doğrular. Bu sayede işlem boyunca iki operandın birbiriyle karışması fiziksel olarak engellenir.

5. Durumların Açıklaması

- `q_baslangic` / `q_yildiz_bulundu` : Başlangıç okumaları ve $=$ işaretini bulma.
- `q_isle_bit_k` : Çarpanın k. bitini okuyup (0 ise x , 1 ise y yazan) karar durumu.
- `q_sola_git_k` / `q_carpan_bul_k` : Referans noktası olan $*$ işaretine geri dönüş durumları.
- `q_shift_basla_k`, `q_shift_elde...` : 1. sayının (çarpılan) sonuna 0 ekleyerek sayıyı bant üzerinde sola kaydıran (physical shift) durum blokları.
- `q_sonuca_git_k` / `q_yaz_k_idx` : 1 okunduğunda, kaydırılmış sayıyı sağdaki $=$ işaretinden sonrasına ekleyen (kopyalayıp toplayan) durumlar.
- `q_kabul` : Tüm bitler işlendiğinde ulaşılan başarılı bitiş durumu.
- `q_red` : Bantta tanımsız bir sembol (Örn: A harfi) veya tanımsız bir duruma gelindiğinde makinenin girdiği ret durumu.

6. Geçiş Mantiğı (Shift & Add)

1. Çarpanın en sağ bitinden başlanır.
2. Eğer okunan bit 0 ise: Sonuca hiçbir şey yazılmaz. Makine doğrudan $*$ işaretine dönerek 1. sayıyı fiziksel olarak bir basamak sola kaydırır (sonuna 0 ekler ve taşıma yapar).

3. Eğer okunan bit 1 ise: Makine = işaretine gider ve 1. sayının mevcut kaydırılmış halini banttaki mevcut toplama ekleyerek yazar. Ardından tekrar * işaretine dönerek 1. sayıyı sonraki adımlar için sola kaydırır.
4. Çarpandaki tüm bitler (X ve Y olarak işaretlendikten sonra) bittiğinde makine q_kabul durumuna geçer.

7. Girdi-Çıktı Örnekleri

Projeye ait 5 farklı test senaryosunun (3x2, 7x3, 2x0, 5x1, 4x3) detaylı adım adım çıktıları, bu Githubdaki Program Çıktıları/test_ornek_X.txt dosyalarında yer almaktadır. Oradaki çıktılar makinenin deterministik adımlarını (state, symbol, tape) şeffafça göstermektedir.

8. Program Ekran Görüntüleri

```
#####
ZORUNLU TASARIM GEREKSİNİMİ: OPERAND AYRIŞTIRMA RAPORU
#####
Turing Makinesi bant üzerindeki verileri aşağıdaki kurallara göre ayrıştırır:
1. '*' karakteri iki sayıyı (multiplicand ve multiplier) ayırmak için kullanılır.
2. '=' karakteri hesaplama sonucunun yazılacağı alanın başlangıcını belirtir.
Bant Ayrıştırma Analizi:
- Tam Bant Girdisi      : 11*10=
- Ayraç (*) Konumu      : İndeks 2
- *'ın Sol Tarafı       : '11' -> Birinci Sayı (Multiplicand)
- *'ın Sağ Tarafı       : '10' -> İkinci Sayı (Multiplier)
- Sonuç Alanı İşareti    : İndeks 5 (= karakterinden sonrası)
Makine işlem boyunca bu ayrımı açık şekilde koruyacak ve her adımda ilgili operand sınırlarını (*) referans alarak işlem yapacaktır.
#####
```

```
Adım 32:
Mevcut Durum      : q_carpan_bul_1
Okunan Sembol     : *
Yazılacak Sembol  : *
Sonraki Yön       : Sağ (R)
Bant İçeriği      : 110*YX=110
Açıklama          : Operand ayrımı (*) teyit edildi. Tüm bitler işlendi, çarpma tamamlandı (q_kabul).

--- SİMÜLASYON BİTTİ ---

SONUÇ EKRANI

Binary Sonuç      : 110
Decimal Sonuç     : 6
Açıklama: 112 (3) × 102 (2) = 1102 (6)
```

9. Sonuç ve Değerlendirme

Geliştirilen bu Turing makinesi, standart bir işlemciadaki çarpma biriminin (multiplier) en temel düzeydeki davranışını tek bir bant üzerinde kusursuz bir şekilde simüle etmektedir. Özellikle "Fiziksel Kaydırma" (Physical Shift) mantığının doğrudan bant üzerine implemente edilmesi ve "Operand Sınırlarının" (* ve =) kafa navigasyonunda aktif olarak kullanılması, projenin tamamen teorik Turing sınırlamaları içinde kalarak hedefe ulaştığını kanıtlamaktadır. İşlem hiçbir bellek sızıntısına veya tanımsız duruma (q_red) mahal vermeden başarıyla tamamlanmaktadır.